

Progress Report

Multi-Agent RAG Educational Chatbot for Women's Mental Health

Architecture, model selection, symptom graph design, edge ranking, and implementation roadmap

Date: 20 June 2026

Executive Summary

This thesis proposes a multi-agent retrieval-augmented generation system for an educational chatbot focused on women's mental health. The system is intended to answer broad user questions about symptoms such as anxiety, depression, fatigue, sleep deprivation, PMS, PMDD, possible hormonal changes, nutrition concerns, and lifestyle factors. The purpose is not to diagnose or prescribe. The purpose is to help users understand possible explanations, learn what evidence exists, identify safe self-care ideas, and know when professional or urgent help is appropriate.

The central design choice is to combine RAG, a symptom graph, and multiple specialized agents. RAG is suitable because the knowledge base will be built from open-access papers, books, and podcast transcripts, and because mental health answers need traceable evidence rather than unsupported generation. The symptom graph is suitable because many women's mental health complaints are connected across domains: poor sleep can worsen anxiety and fatigue; premenstrual symptoms can include sleep, appetite, mood, and concentration changes; stress can interact with cycle symptoms; and interventions such as exercise, nutrition, sleep hygiene, light exposure, psychotherapy, or supplements must be considered with contraindications and strength of evidence.

The proposed architecture should use eight agents: Router Agent, Safety Agent, Sleep Agent, Hormonal Health Agent, Nutrition and Body Agent, Lifestyle Context Agent, Intervention Agent, and Synthesizer Agent. The original list combined lifestyle and intervention, but for a thesis system it is cleaner to separate them: the Lifestyle Context Agent interprets user context, while the Intervention Agent ranks possible actions and checks that they are appropriate for the symptoms, evidence level, risks, medications, pregnancy/lactation, and user constraints.

Recommended first prototype: Start with a baseline RAG system using Qwen2.5-7B-Instruct or Qwen3-8B as the local/free generator, a SentenceTransformers embedding model for retrieval, a cross-encoder reranker for confidence, and DeBERTa-v3 or MentalBERT/MentalRoBERTa for classification experiments. Implement the graph and ranking logic before attempting fine-tuning.

1. Thesis Aim and Research Rationale

The thesis problem is meaningful because women's mental health questions rarely fit into one clinical category. A user may ask: "I feel exhausted, anxious, and cannot sleep before my period. What could be happening?" A simple chatbot might answer only about anxiety or only about sleep. A better educational system should consider several plausible domains at once: sleep quality, mood symptoms, menstrual timing, premenstrual disorders, nutrition, stress, exercise, medical red flags, and whether the user should speak with a healthcare professional.

A multi-agent RAG design supports this because each agent can retrieve from a domain-specific subcorpus and reason within a narrower conceptual space. The Sleep Agent can focus on insomnia, circadian rhythm, sleep hygiene, fatigue, sleep deprivation, and sleep disorders. The Hormonal Health Agent can focus on menstrual cycle timing, PMS, PMDD, perimenopause, thyroid-related educational material, postpartum considerations, and symptom tracking. The Nutrition and Body Agent can focus on food patterns, body image risk, deficiencies, digestive symptoms, exercise, supplements, and medication/supplement cautions. The Safety Agent can detect self-harm, eating disorder risk, severe depression, pregnancy-related red flags, medication danger, or urgent medical symptoms. The Synthesizer Agent then integrates evidence from the specialists into one answer.

The chatbot should be framed as educational. It should answer in language such as "possible explanations include" rather than "you have." It should encourage symptom tracking, professional consultation when symptoms impair daily life, and emergency/crisis support for self-harm, suicidal thoughts, acute confusion, chest pain, severe bleeding, or other urgent signs. This framing is not only ethically necessary; it also improves the scientific validity of the thesis because the system can be evaluated on evidence retrieval, citation quality, uncertainty handling, and safe escalation rather than on clinical diagnosis.

The project should also define population scope carefully. The phrase "women's mental health" is understandable for the thesis topic, but the system should avoid assuming that all users have menstrual cycles or that only women have menstrual or hormonal health questions. A precise implementation can say that it focuses on women's mental health and reproductive-hormonal factors while remaining inclusive of people whose symptoms or bodies make the information relevant.

2. Proposed Eight-Agent Architecture

The agents do not need to be eight separate large models. A practical thesis implementation can use one local instruction model with different prompts, tools, retrievers, and schemas for each agent. This reduces memory cost while preserving modular reasoning. Each agent receives the user query, a normalized user profile if available, relevant retrieved passages, and graph candidates. Each agent returns a structured object: domain assessment, top concepts, retrieved evidence IDs, confidence score, uncertainty notes, and safety flags.

Agent	Main responsibility	Inputs	Outputs
Router Agent	Classifies the query into domains and decides which	Query text, optional profile, previous turn	Domain labels, agent plan, query reformulations,

Agent	Main responsibility	Inputs	Outputs
	specialist agents should run.	summary.	confidence.
Safety Agent	Runs before and after generation to detect crisis, self-harm, eating disorder danger, medication/supplement danger, pregnancy red flags, or urgent medical symptoms.	Raw query, retrieved evidence, draft answer.	Safety level, escalation language, blocked/allowed answer scope.
Sleep Agent	Handles insomnia, sleep deprivation, fatigue, circadian disruption, nightmares, shift work, and sleep-hygiene education.	Sleep corpus, CDC/NHLBI-style facts, symptom graph nodes for sleep/fatigue/mood.	Sleep-related possible explanations, sleep diary suggestions, red flags.
Hormonal Health Agent	Handles PMS, PMDD, cycle timing, perimenopause, postpartum, thyroid-related educational links, and symptom tracking around hormonal changes.	Cycle/hormonal sources, menstrual symptom graph, age/life-stage context.	Hormonal hypotheses, tracking questions, when to seek care.
Nutrition and Body Agent	Handles diet patterns, cravings, caffeine, iron/vitamin considerations, body image, exercise, digestive overlap, and supplement safety.	Nutrition/supplement sources, contraindication nodes, body-related symptoms.	Food/lifestyle education, supplement cautions, body-image/eating-risk flags.
Lifestyle Context Agent	Interprets demographic and lived-context variables such as age, sleep schedule, stress, work, culture, access to food/exercise, family history, and social support.	User-provided context only; no hidden inference.	Context modifiers, missing context questions, equity/access notes.
Intervention Agent	Connects symptoms and likely contributing factors to possible interventions while ranking evidence and checking contraindications.	Candidate symptoms, graph edges, intervention library, evidence scores.	Ranked interventions with evidence grade, risks, and suitability.
Synthesizer Agent	Combines specialist outputs into one coherent answer with citations, uncertainty, next steps, and safety guidance.	All agent outputs, final retrieved passages, safety constraints.	Final response, citations, confidence summary, follow-up questions.

The Router Agent is important because not every query should activate every specialist. For example, "I cannot sleep and I feel sad before my period" should route to Sleep, Hormonal Health, Lifestyle Context, Intervention, Safety, and Synthesizer. A question such as "Can magnesium help PMS?" should route to

Nutrition and Body, Hormonal Health, Intervention, Safety, and Synthesizer. A query such as "I want to disappear" should immediately prioritize Safety, with the chatbot providing crisis-oriented support instead of normal educational RAG.

The Safety Agent should be a gatekeeper, not an afterthought. It should run at three points: first on the raw user message, second on proposed retrieved content or intervention candidates, and third on the final answer. This is especially important for women's mental health because the system may discuss self-harm, depression, PMDD, eating disorders, supplements, pregnancy/lactation, medication interactions, trauma, or domestic safety. The Safety Agent can allow educational discussion while preventing the chatbot from giving diagnosis, dosing, dangerous supplement combinations, or advice that delays urgent care.

3. Symptom Graph and Concept Design

The symptom graph is the thesis component that can make the system more than a normal RAG chatbot. A vector database retrieves text chunks, but a graph represents relationships: symptoms can co-occur, symptoms may worsen each other, conditions can explain symptom clusters, and interventions can target mechanisms or behaviors. This is useful for women-centered mental health because many questions involve interacting systems rather than one isolated symptom.

Each agent can maintain a domain concept set, but the graph should connect across agents. For example, the Sleep Agent may have concepts such as insomnia, delayed sleep phase, poor sleep quality, fatigue, nightmares, caffeine, shift work, and rumination. The Hormonal Health Agent may have luteal phase, PMS, PMDD, dysmenorrhea, perimenopause, postpartum, estrogen/progesterone change, and symptom tracking. The Nutrition and Body Agent may have caffeine, iron, B6, calcium, magnesium, cravings, low energy availability, body image distress, and supplement interaction. The Intervention Agent connects these to possible interventions such as aerobic exercise, sleep schedule stabilization, light exposure, CBT-I principles, mindfulness, nutrition changes, calcium or B6 education, clinician evaluation, or crisis support.

A useful graph schema can include the following node types: Symptom, Symptom Cluster, Condition/Educational Topic, Life Stage, Risk Factor, Protective Factor, Intervention, Contraindication, Evidence Source, Demographic Context, and Follow-up Question. It should include edges such as `co_occurs_with`, `may_be_associated_with`, `may_worsen`, `may_improve`, `has_red_flag`, `contraindicated_for`, `supported_by_source`, `requires_professional_evaluation`, and `asks_about`. Every edge should store provenance: source ID, passage ID, extraction method, agent that proposed it, date extracted, and a confidence score.

- Example graph path: sleep deprivation -> may_worsen -> anxiety; anxiety -> may_worsen -> insomnia; insomnia -> may_improve_with -> sleep schedule regularity; sleep schedule regularity -> supported_by_source -> CDC/NHLBI sleep guidance.
- Example graph path: luteal phase symptoms -> may_be_associated_with -> PMS/PMDD; PMS/PMDD -> may_include -> sleep problems, fatigue, appetite changes, anxiety, depression-like symptoms; PMDD severity -> requires_professional_evaluation -> clinician/mental health professional.

- Example graph path: PMS symptoms -> may_improve_with -> aerobic physical activity, sleep, stress coping; chasteberry -> contraindicated_for -> hormonal birth control or hormone therapy according to patient-facing federal health material; supplement recommendation -> requires_safety_note -> discuss with clinician.

The graph should not claim causality unless the source explicitly supports causality. Many relationships in mental health are associative or bidirectional. Therefore, edge labels should be careful: may_be_associated_with is safer than causes; may_worsen is safer than always worsens; may_improve_with is safer than cures. This language also improves final answer quality because the Synthesizer Agent can communicate uncertainty honestly.

4. Ranking and Confidence Methods

Ranking is essential because a graph extracted from papers, books, and transcripts can become noisy. The thesis should explicitly avoid showing every extracted relationship. Instead, the system should only use edges and interventions that pass confidence thresholds. Ranking should happen at three layers: retrieval ranking, graph-edge ranking, and answer/intervention ranking.

Retrieval ranking should begin with hybrid retrieval. Dense embeddings are good for semantic similarity, but keyword search is still useful for medical terms, abbreviations, supplement names, and symptoms. A good first system retrieves top-k passages with BM25 or a sparse retriever plus top-k passages with embeddings, merges the results, removes duplicates, and reranks the final candidates with a cross-encoder. SentenceTransformers documentation explicitly supports a retrieve-and-rerank pattern where a fast bi-encoder retrieves candidates and a slower cross-encoder reranks query-passage pairs. This is appropriate for a thesis because it gives measurable retrieval metrics such as Recall@k, MRR, and nDCG.

Graph-edge ranking should score an edge by semantic support, source quality, cross-source agreement, extraction confidence, agent agreement, recency when relevant, contradiction penalty, and safety penalty. A possible first formula is:

```
edge_score =
  0.30 * source_quality
+ 0.25 * semantic_support
+ 0.15 * cross_source_agreement
+ 0.10 * extraction_confidence
+ 0.10 * agent_consensus
+ 0.05 * recency_or_guideline_status
- 0.20 * contradiction_penalty
- 0.15 * safety_penalty
```

This formula is intentionally transparent. It can be tuned during evaluation. Source quality can rank clinical guidelines, systematic reviews, official health sources, peer-reviewed papers, textbooks, and transcripts differently. Semantic support can come from the reranker score between the edge statement and the supporting passage. Cross-source agreement increases when multiple independent sources support the same relationship. Contradiction penalty increases when high-quality sources disagree or when the evidence is mixed. Safety penalty lowers edges involving supplements, pregnancy, psychiatric crisis, medication changes, eating-disorder content, or urgent medical symptoms unless the answer includes appropriate caution.

Intervention ranking should be more conservative than symptom explanation ranking. It is acceptable to say that poor sleep, PMS, stress, or depression can overlap. It is more risky to suggest an herb, supplement, or behavior change. Therefore, the Intervention Agent should only show interventions if the evidence score is high enough and no major contraindication is detected. It should group interventions into low-risk educational suggestions, clinician-discussion topics, and urgent-care/crisis actions.

- High-confidence edge threshold: use in final answer and cite source.
- Medium-confidence edge threshold: mention as a possibility with uncertainty language or ask a follow-up question.
- Low-confidence edge threshold: keep internally for graph exploration but do not include in user answer.
- Safety-sensitive threshold: require Safety Agent approval and a caution note even if evidence score is high.

5. Data Sources, Q&A Extraction, and Knowledge Base

The collected open-access resources are a strong foundation, but they need a disciplined data pipeline. Papers, books, and podcast transcripts are not equal evidence. The system should store metadata for every source: title, author, year, license, source type, URL/DOI, domain, quality tier, whether it is peer-reviewed, whether it is patient-facing or technical, and whether reuse is allowed. Podcast transcripts can be valuable for natural phrasing and Q&A examples, but they should usually have lower evidence weight than guidelines, systematic reviews, or official health resources.

The Q&A extraction plan should not simply generate many question-answer pairs and treat them all as truth. Each generated Q&A pair should include the supporting passage IDs, answer span, domain tags, safety tags, and whether the answer is extractive or synthesized. A good rule is: if the answer cannot point to a supporting passage, discard it or send it to manual review. This keeps the dataset useful for evaluation and possible fine-tuning later.

1. Convert each resource into clean text while preserving section headings, page numbers, timestamps, tables, and references where possible.
2. Chunk text into semantically coherent passages, for example 250-500 tokens with overlap, while keeping source metadata.
3. Generate candidate Q&A pairs from each chunk with a structured prompt that asks for question, answer, domain labels, evidence span, safety flags, and concepts.
4. Validate Q&A pairs by checking that answer text is supported by the chunk and that the answer does not overstate evidence.
5. Use accepted Q&A pairs for evaluation first, not for model fine-tuning first. Fine-tuning should come later only if baseline RAG is insufficient.
6. Extract candidate graph concepts and edges from the same chunks, then rank and deduplicate them before adding to the symptom graph.

Data governance: For demographic, family-history, and genetic-like information, collect only what the user voluntarily provides and only what is needed for the answer. The chatbot should not infer sensitive attributes, should not store identifiable personal health information unless the thesis explicitly has ethics approval, and should treat family history as contextual risk information rather than deterministic prediction.

6. Model Selection: What to Use and Why

A key clarification for the thesis is that MentalBERT, MentalRoBERTa, and DeBERTa are not chat LLMs for generating long user-facing answers. They are encoder or masked-language models, excellent for classification, embeddings, reranking, or relation extraction after fine-tuning. The final answer generator should be an instruction-tuned causal language model such as Qwen, Mistral, Llama, Phi, or Gemma. The best architecture is therefore a model stack, not one model for everything.

The model stack should separate tasks: an embedding model retrieves relevant passages; a reranker orders them; classifier models detect domains, symptoms, risks, and graph relations; and a generator writes the final response grounded in retrieved evidence. This separation makes the thesis easier to evaluate and safer than relying on one LLM to classify, retrieve, reason, and answer without checks.

Model	License/access	Pros	Cons	Thesis use
Qwen2.5-7B-Instruct	Apache 2.0; open-weight	Strong instruction following, JSON/structured output, long context up to 128K according to model card, good first RAG generator.	7B still needs GPU or 4-bit quantization; not medical-specialized.	Best first baseline if hardware can run 7B in 4-bit.
Qwen3-8B	Apache 2.0; open-weight	Newer Qwen family, strong reasoning, tool/agent capability, 100+ languages, thinking/non-thinking switch.	Thinking mode adds complexity; native context is shorter than Qwen2.5 unless using long-context configuration.	Second experiment or primary model if reasoning/tool use is central.
Phi-4-mini-instruct	MIT; open model	Only 3.8B parameters, 128K context, strong for constrained hardware and RAG settings.	Smaller factual capacity; may need stronger retrieval and reranking.	Best low-resource fallback or CPU/limited-GPU prototype.
Mistral-7B-Instruct	Apache 2.0 for listed v0.3 model card	Mature 7B option, widely supported by local inference tools.	Older than Qwen3/Phi-4-mini; model card warns many instruct models lack domain-specific guardrails.	Useful comparison baseline.

Model	License/accesses	Pros	Cons	Thesis use
Llama 3.1 8B Instruct	Custom Llama 3.1 Community License	Strong multilingual chat model, 128K context, broad ecosystem.	License is not Apache/MIT; gated access; acceptable-use terms must be checked carefully for health use.	Good benchmark, but less ideal as first choice if license simplicity matters.
MentalBERT / MentalRoBERTa	CC BY-NC 4.0; gated	Domain-specific mental health encoders trained on mental-health Reddit posts; useful for mental-health classification experiments.	Non-commercial license, gated access, trained on social media, not diagnostic, not a generator.	Use for research classifiers, not final answer generation.
DeBERTa-v3-base	MIT	Strong encoder for classification, relation extraction, reranking, and NLU tasks; efficient compared with LLMs.	Requires labeled data for thesis-specific tasks; not a chat generator.	Good starting model for domain routing and edge extraction classifiers.
Gemma 3 IT	Google Terms for Gemma 3; open weights	Multimodal family, long context, strong general tasks.	License/terms are less simple than Apache/MIT for earlier Gemma 3; access may require acceptance.	Optional comparison, not first baseline.

My recommended starting point is Qwen2.5-7B-Instruct in 4-bit quantization for the first complete prototype, with Phi-4-mini-instruct as the lightweight fallback and Qwen3-8B as the second experiment. Qwen2.5-7B-Instruct is a practical first choice because the thesis needs reliable structured outputs, long-context handling, and simple RAG behavior before adding reasoning-mode complexity. Qwen3-8B should still be included in the evaluation because it is newer and explicitly positioned for agentic use. If hardware is limited, Phi-4-mini-instruct is the most pragmatic first model because it is MIT licensed and designed for memory/compute-constrained environments.

MentalBERT should not be discarded. It can be valuable for detecting mental-health-related language, grouping symptoms, or comparing domain-specific encoders against general encoders. However, because it is CC BY-NC and gated, and because its model card emphasizes that predictions are not psychiatric diagnoses, it should be described as a research classifier or feature extractor, not the main chatbot LLM. DeBERTa-v3 is a stronger license option for classifiers because it is MIT licensed and widely used for NLU fine-tuning.

7. Implementation Roadmap and Coding Steps

The coding should proceed in layers. The first milestone should be a single-agent baseline RAG system. The second milestone should add routing and specialist agents. The third milestone should add graph

extraction and edge ranking. The fourth milestone should add evaluation and safety testing. This order reduces risk because it creates a working system early and allows each additional component to be evaluated against the previous baseline.

7.1 Recommended Libraries

- Core ML: torch, transformers, accelerate, bitsandbytes for 4-bit quantized local model loading.
- Retrieval: sentence-transformers for embeddings and cross-encoder reranking; rank-bm25 for keyword search; ChromaDB or FAISS for vector storage.
- RAG orchestration: LangChain/LangGraph or LlamaIndex. LangGraph is useful if the agents are modeled as a state machine; LlamaIndex is convenient for ingestion and query engines.
- Graph: networkx for first prototype; Neo4j if the graph becomes large or if graph queries need to be central to the thesis demo.
- Data processing: pandas, pydantic, pdfplumber or PyMuPDF, BeautifulSoup4, datasets, evaluate, scikit-learn.
- Backend/demo: FastAPI for an API, Streamlit or Gradio for a supervisor-facing prototype.
- Evaluation: ragas or custom metrics for faithfulness/citation coverage, plus sklearn metrics for classifiers and manual expert review spreadsheets.

7.2 Environment and Model Download

A minimal environment can be created with Python 3.10+ and PyTorch. Hugging Face Transformers downloads models through `from_pretrained` and caches them locally. Gated models such as MentalBERT or Llama require accepting the model terms and authenticating with a Hugging Face token. Open models such as Qwen and Phi can usually be loaded directly, subject to disk and memory.

```
python -m venv .venv
source .venv/bin/activate
pip install torch transformers accelerate bitsandbytes sentence-transformers
pip install chromadb faiss-cpu rank-bm25 langchain langgraph pydantic fastapi uvicorn
pip install pandas scikit-learn pdfplumber pymupdf networkx
```

```
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig

model_name = "Qwen/Qwen2.5-7B-Instruct"
quantization_config = BitsAndBytesConfig(load_in_4bit=True)
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    device_map="auto",
    torch_dtype="auto",
    quantization_config=quantization_config,
)
```

7.3 Suggested Repository Structure

```
thesis-rag-chatbot/
  data/raw/           # PDFs, transcripts, books
  data/processed/     # cleaned text, chunks, metadata
  data/qa_pairs/      # extracted and reviewed Q&A pairs
```

```

app/agents/           # router, safety, domain agents, synthesizer
app/retrieval/        # embedding, hybrid search, reranking
app/graph/            # schema, extraction, edge ranking
app/prompts/          # system prompts and structured templates
app/evaluation/       # retrieval, graph, answer, and safety tests
app/api/              # FastAPI or demo interface
notebooks/            # experiments and error analysis
tests/                # unit and integration tests

```

7.4 Core Functions

Function	Purpose
<code>ingest_documents(source_dir)</code>	Load PDFs, text files, transcripts, and metadata into a standardized Document object.
<code>clean_text(document)</code>	Remove headers/footers/noise, preserve section titles, normalize whitespace, and flag tables.
<code>chunk_document(document)</code>	Create overlapping chunks with source ID, page/timestamp, heading, license, and domain metadata.
<code>embed_chunks(chunks)</code>	Compute embeddings and store them in Chroma/FAISS with metadata filters.
<code>hybrid_retrieve(query, filters)</code>	Combine BM25 and vector retrieval before reranking.
<code>rerank(query, candidate_chunks)</code>	Use a cross-encoder to produce relevance scores for top candidate passages.
<code>extract_concepts(chunk)</code>	Identify symptoms, conditions, lifestyle factors, interventions, contraindications, and source evidence.
<code>score_edge(edge, evidence)</code>	Assign confidence based on source quality, semantic support, agreement, contradiction, and safety.
<code>route_query(query, profile)</code>	Select the domain agents and rewrite the query for each selected retriever.
<code>run_agent(agent_name, query, evidence, graph_context)</code>	Return structured agent output with confidence and citations.
<code>safety_check(query_or_answer)</code>	Classify risk level and decide whether to answer normally, add caution, or escalate.
<code>synthesize_answer(agent_outputs)</code>	Generate a final educational answer with uncertainty, citations, and next steps.

A simple state object can hold the full multi-agent workflow. Each agent should read from and write to the same state rather than passing unstructured text between agents. This makes the system auditable and easier to evaluate.

```

class RAGState(BaseModel):
    user_query: str

```

```

user_profile: dict = {}
route: list[str] = []
retrieved_chunks: dict[str, list[EvidenceChunk]] = {}
graph_candidates: list[GraphEdge] = []
agent_outputs: dict[str, AgentOutput] = {}
safety_level: str = "normal"
final_answer: str | None = None

```

8. Answer Design for the Educational Chatbot

The final answer should be structured but not overwhelming. For most health questions, a useful answer pattern is: acknowledge the concern; list the most plausible educational explanations; identify red flags; suggest low-risk next steps; explain when to consult a professional; and cite the evidence used. The Synthesizer Agent should avoid unsupported claims and avoid presenting interventions as guaranteed solutions.

For example, if a user asks about anxiety, insomnia, fatigue, and symptoms before menstruation, the chatbot can say that several patterns may overlap: sleep deficiency can worsen mood and attention; PMS can include sleep problems, appetite changes, fatigue, anxiety, mood swings, and depressive feelings; PMDD is a more severe form that should be discussed with a clinician if symptoms impair daily life; and depression/anxiety can also worsen around the premenstrual period. The answer can recommend tracking symptoms across at least several cycles, reviewing sleep regularity and caffeine timing, considering gentle aerobic activity if safe, and seeking professional help if symptoms are severe, persistent, or include self-harm thoughts.

For intervention answers, the system should separate low-risk lifestyle education from clinician-discussion topics. Sleep regularity, exercise, stress coping, and symptom tracking can be presented as general educational strategies. Supplements such as calcium, B6, magnesium, chasteberry, black cohosh, or evening primrose oil require more caution because supplements can interact with medication or be inappropriate in pregnancy, lactation, hormone therapy, or certain conditions. The chatbot should not give dosing instructions unless the source and scope are appropriate, and even then it should frame supplement use as something to discuss with a healthcare professional.

Crisis response: If the user expresses suicidal thoughts, self-harm intent, inability to stay safe, or severe crisis, the chatbot should stop normal RAG answering and provide supportive crisis language plus local emergency guidance. In the United States, 988 is available by call, text, or chat 24/7/365 for mental health or emotional distress.

9. Evaluation Plan

The evaluation should be thesis-driven rather than only demo-driven. It should answer whether the multi-agent graph-based RAG system improves quality over a simpler baseline. The baseline can be: one retriever plus one generator, no agents, no graph, no reranker. The proposed system can then be compared against this baseline on retrieval quality, answer faithfulness, safety behavior, and usefulness.

- Retrieval metrics: Recall@5/10, MRR, nDCG, citation coverage, and whether the cited passages actually support the answer.

- Graph metrics: precision of extracted edges, duplicate rate, contradiction rate, and expert/manual rating of high-confidence edges.
- Agent metrics: routing accuracy, specialist answer relevance, confidence calibration, and whether agents disagree in useful ways.
- Generation metrics: faithfulness to retrieved evidence, clarity, uncertainty expression, absence of diagnosis, and answer completeness.
- Safety metrics: correct handling of self-harm, eating disorder risk, supplement risks, pregnancy/lactation cautions, medication changes, and urgent symptoms.
- User-centered metrics: perceived helpfulness, readability, and whether the answer encourages appropriate professional care when needed.

A practical evaluation set can be created from the extracted Q&A pairs plus manually written scenario prompts. Scenario prompts should include normal educational questions, ambiguous multi-symptom questions, demographic-context questions, supplement questions, crisis prompts, and adversarial prompts that try to make the system diagnose or give unsafe instructions. Each answer can be rated with a rubric from 1 to 5 for evidence support, safety, clarity, and relevance.

10. Risks, Ethics, and Thesis Limitations

The main technical risk is hallucination: the generator may produce confident claims not supported by the retrieved sources. RAG reduces this risk but does not eliminate it. This is why source-grounded generation, citation checking, conservative answer templates, and reranking are necessary. The second technical risk is graph noise: automatic relation extraction can create false symptom-intervention links. This is why the graph should store provenance, confidence, and contradiction flags.

The main ethical risk is that users may treat an educational chatbot as medical authority. The interface and answer style must repeatedly make clear that it is not a doctor, therapist, or emergency service. It should not diagnose PMDD, depression, anxiety disorders, thyroid disease, anemia, PCOS, or any other condition. It can explain patterns, encourage tracking, and recommend professional evaluation when symptoms are severe, persistent, worsening, or impairing daily life.

Demographic and family-history personalization must be handled carefully. Age, menstrual status, pregnancy/postpartum status, perimenopause, work schedule, caregiving burden, cultural food access, and family history may affect the relevance of an answer. However, this context should be optional, user-controlled, and used only to tailor education. The system should not infer genetic risk or make deterministic claims from family history. For a thesis prototype, storing personal data should be avoided unless there is a clear ethics protocol.

Bias is another limitation. Mental health language differs by culture, age, socioeconomic background, neurodivergence, trauma history, and language. A model trained mainly on English internet data may misunderstand some users. The evaluation should therefore include diverse phrasing and should test whether the system gives equitable, non-stigmatizing responses. The chatbot should not shame users about weight, food, exercise, menstruation, medication, or mental health.

11. Proposed Milestones

Milestone	Name	Work	Output
1	Corpus preparation	Clean sources, create metadata schema, chunk documents, build initial vector store.	Processed corpus and source catalog.
2	Baseline RAG	Implement single-agent RAG with citations and safety disclaimer.	Working baseline chatbot.
3	Agent routing	Add Router, Safety, Sleep, Hormonal, Nutrition/Body, Lifestyle, Intervention, Synthesizer prompts.	Structured multi-agent workflow.
4	Symptom graph	Extract concepts/edges, create graph schema, implement edge scoring and graph retrieval.	Ranked symptom-intervention graph.
5	Evaluation	Build test set, compare baseline vs multi-agent graph RAG, run safety red-team prompts.	Evaluation tables and error analysis.
6	Thesis writing	Document methodology, results, limitations, ethics, and future work.	Final thesis chapters and demo.

The most important next step is to build the baseline RAG and evaluation set before spending too much time on fine-tuning. A strong baseline will make it possible to prove whether the multi-agent graph design adds value. Fine-tuning should be treated as an optional later improvement, not the first implementation step.

12. References and Source Anchors

The following sources were used to ground the technical choices, model comparisons, and safety framing. They should be treated as source anchors for the progress report rather than a complete literature review.

7. [Hugging Face Transformers installation and model caching](#)
8. [Hugging Face Transformers text generation and quantization example](#)
9. [Hugging Face bitsandbytes quantization documentation](#)
10. [SentenceTransformers quickstart and retrieve/rerank documentation](#)
11. [SentenceTransformers cross-encoder applications](#)
12. [LangChain RAG tutorial](#)
13. [Qwen3-8B model card](#)
14. [Qwen2.5-7B-Instruct model card](#)
15. [Phi-4-mini-instruct model card](#)
16. [Mistral-7B-Instruct-v0.3 model card](#)
17. [Llama 3.1 8B Instruct model card and license](#)
18. [MentalBERT model card](#)

19. [MentalRoBERTa model card](#)
20. [DeBERTa-v3-base model card](#)
21. [WHO ethics and governance of AI for health](#)
22. [CDC About Sleep](#)
23. [NHLBI Sleep Deprivation and Deficiency](#)
24. [Office on Women's Health PMS and PMDD information](#)
25. [NIH Office of Dietary Supplements fact sheets](#)

Date: 25th of June

Part 1: Deconstructing the Baseline RAG System

1. Hardware Optimization & The Generator (Cell 2)

- **What it does:** We are loading **Qwen2.5-7B-Instruct**, an extremely capable open-source language model. However, a 7 billion parameter model normally requires ~14GB of VRAM just to load, which would crash a free Google Colab instance.
- **The Technical Detail:** We use `BitsAndBytesConfig(load_in_4bit=True)`. This performs **4-bit quantization**, mathematically compressing the model's weights on the fly. It shrinks the memory footprint to about 5GB, allowing it to run comfortably on a Colab T4 GPU while preserving its reasoning capabilities. We also set the `temperature` to 0.1 to strictly limit creative hallucination—in healthcare contexts, we want factual precision, not storytelling.

2. The Knowledge Engine (Cell 3)

- **What it does:** This is where we ingest our medical texts and podcast transcripts.
- **The Technical Detail:** We use a `RecursiveCharacterTextSplitter` to break documents into chunks of 300 characters, with a 50-character overlap. This overlap ensures a sentence isn't abruptly cut in half, losing its context. These chunks are converted into mathematical vectors using a lightweight embedding model (`all-MiniLM-L6-v2`) and stored in **ChromaDB**. When a user asks a question, the system plots the question in the same vector space and retrieves the top 3 nearest text chunks (`k=3`).

3. Conversational Memory - The History-Aware Retriever (Cell 4)

- **What it does:** Standard RAG fails miserably at follow-up questions. If a user asks "What is PMDD?" and then asks "What supplements help with **it**?", a standard vector search for the word "it" will retrieve useless garbage.
- **The Technical Detail:** We solve this using a two-step LLM chain. The first prompt (`contextualize_q_system_prompt`) intercepts the follow-up question, looks at the chat history, and asks the LLM to rewrite it. "What supplements help with it?" is silently rewritten into "What supplements help with PMDD?" *before* it searches the database.

4. The Educational Guardrail (Cell 4)

- **What it does:** We strictly enforce the ethical boundaries of your thesis.
- **The Technical Detail:** The `qa_system_prompt` acts as the system's "constitution." By hardcoding instructions like *"You are NOT a doctor... Use language such as 'possible explanations include'"*, we force the LLM to synthesize the retrieved chunks through an educational lens rather than a diagnostic one.

Part 2: The Evolution into the Multi-Agent System

1. Moving from LangChain to LangGraph

- Currently, our code uses linear `chains` (A -> B -> C). To build agents, we will upgrade to **LangGraph**, which allows for cyclical, state-machine workflows (A -> B, B realizes it needs more info, loops back to A).
- We will define a global `State` dictionary that contains the user's query, their demographic profile, safety flags, and a list of retrieved evidence. All agents will pass this single `State` object around, modifying it as they work.

2. Building the Router & Safety Gatekeepers

- Instead of sending the user's prompt straight to the LLM, it will hit our **Safety Agent** first. This will be a fast, fine-tuned classifier (like the DeBERTa model mentioned in your thesis) that scans for crisis language.
- If safe, the query moves to the **Router Agent**. The Router acts as guide. Using a structured output prompt, it will analyze a query like *"I can't sleep before my period"* and output `["Sleep_Agent", "Hormonal_Agent"]`.

3. Domain-Specific Worker Agents

- We will split our single ChromaDB into multiple distinct collections: one for Sleep, one for Nutrition, one for Mental Health.
- We will wrap our LLM into specialized functions (e.g., `def sleep_agent(state):`). If the Router triggers the Sleep Agent, it will *only* search the sleep-specific vector database and *only* use a sleep-specific system prompt.
- **Crucially**, this allows us to run these agents in **parallel**. The Sleep Agent, Hormonal, Mental Health, Lifestyle Agent can do their research at the exact same time.

4. The Intervention & Synthesizer Agents

- Once the worker agents gather their domain-specific evidence, they append it to the shared `State`.
- The **Intervention Agent** will look at the proposed solutions and query your Symptom Knowledge Graph (using `NetworkX`) to filter out weak evidence or harmful interactions (e.g., flagging a supplement interaction).
- Finally, the **Synthesizer Agent** receives the giant pile of vetted context and writes the final, empathetic, educational response to the user.